

Rapport J12 - DISTINCT, expressions et CASE

1. Informations générales

- Programme : Plan d'exécution SQL/Data - 16 semaines
- Semaine : 02
- Jour : J12
- Date : 26 septembre 2026
- Base : `sql\_training`
- Table : `raw.training\_events`
- PostgreSQL : 17.5
- Durée prévue : 90 minutes
- Durée réelle effectuée : 62 minutes
- Timer arrêté avec 28 minutes restantes
- Durée réelle finale : 62 minutes
- Écart prévu/réel final : -28 minutes (séance terminée 28 minutes avant la durée prévue)

2. Objectifs

J12 vise à transformer les résultats SQL en information métier :

- extraire des valeurs uniques avec `DISTINCT`;
- comprendre `DISTINCT` sur plusieurs colonnes;
- construire des colonnes calculées;
- utiliser `ROUND`;
- classer des données avec `CASE`;
- traiter explicitement les valeurs `NULL`;
- combiner `CASE`, `GROUP BY` et `COUNT(\*)`;
- construire un challenge métier avec `IN`, `ORDER BY` et `LIMIT`;
- produire un script reproductible validé par `psql -f`.

3. Validation initiale

État validé :

- 15 lignes;
- 4 systèmes sources;
- 6 types d'événements;
- `MIN(event\_id) = 1`;
- `MAX(event\_id) = 18`;
- dépôt Git initial propre et synchronisé;
- commit précédent J11 : `2758aa2`.

Capture : `J12-01\_environment-dataset-validation.png`

4. DISTINCT sur une colonne

`SELECT DISTINCT source\_system` retourne 4 sources :

- `manual\_psql`
- `microgrid\_poc`
- `sensor\_gateway`
- `tomra\_training`

`SELECT DISTINCT event\_type` retourne 6 types :

- `current`
- `machine\_alarm`
- `sensor\_reading`
- `temperature`
- `training\_test`
- `voltage`

`DISTINCT` ne supprime aucune donnée de la table : il élimine les doublons du résultat du `SELECT`.

Capture : `J12-02\_distinct-single-column.png`

5. DISTINCT multi-colonnes

La combinaison :

```
SELECT DISTINCT
    source_system,
    event_type
FROM raw.training_events
ORDER BY source_system, event_type;
```

retourne 6 couples distincts.

Le contrôle par sous-requête confirme :

```
distinct_source_event_pairs = 6
```

`DISTINCT` s'applique à la combinaison des colonnes sélectionnées, et non indépendamment à chacune.

Capture : `J12-03\_distinct-multi-column.png`

6. Expressions et colonnes calculées

Expression testée :

```
event_value * 1.10 AS value_plus_10_percent
```

Puis :

```
ROUND(event_value * 1.10, 2) AS value_plus_10_percent
```

La première expression conserve davantage de décimales, par exemple `333.5750`. `ROUND(..., 2)` produit `333.58`.

L'alias nomme une colonne calculée du résultat; aucune colonne physique n'est ajoutée à la table.

Capture : `J12-04\_calculated-column-round.png`

7. CASE simple

Règle :

- `event\_value >= 200` -> `HIGH`
- sinon -> `STANDARD`

Résultat observé : les cinq valeurs supérieures ou égales à 200 sont classées `HIGH`; les autres sont `STANDARD`.

Capture : `J12-05\_case-basic-classification.png`

8. CASE multi-niveaux

Règles :

- `>= 200` -> `HIGH`
- `>= 25` -> `MEDIUM`
- sinon -> `LOW`

Résultats notables :

- 315.90 -> HIGH
- 125.50 -> MEDIUM
- 31.40 -> MEDIUM

- 24.80 -> LOW
- 1.00 -> LOW

L'ordre des `WHEN` est essentiel lorsque les conditions se chevauchent : la condition la plus restrictive doit être évaluée avant la plus générale.

Capture : `J12-06\_case-multi-level-classification.png`

9. Gestion explicite de NULL

La logique robuste utilisée est :

```
CASE
  WHEN event_value IS NULL THEN 'MISSING'
  WHEN event_value >= 200 THEN 'HIGH'
  WHEN event_value >= 25 THEN 'MEDIUM'
  ELSE 'LOW'
END
```

Aucune ligne actuelle n'est `NULL`, donc `MISSING` n'apparaît pas dans le résultat. La branche est néanmoins explicitement prévue pour de futures données manquantes.

Capture : `J12-07\_case-null-handling.png`

10. KPI par catégorie CASE

Résultat :

Contrôle d'intégrité logique : `6 + 5 + 4 = 15`.

Cette étape combine transformation métier et agrégation :

`CASE -> catégorie -> GROUP BY -> COUNT(\*) -> KPI`.

Capture : `J12-08\_case-grouped-kpi.png`

11. Challenge autonome

Besoin :

- colonnes `event\_id`, `source\_system`, `event\_type`, `event\_value`, `priority`;
- `NULL` -> `UNKNOWN`;
- `>= 200` -> `CRITICAL`;
- `>= 25` -> `REVIEW`;
- sinon -> `NORMAL`;
- sources limitées à `microgrid\_poc` et `sensor\_gateway`;
- tri `event\_value DESC`, puis `event\_id ASC`;
- maximum 8 lignes.

Requête réussie :

```
SELECT
  event_id,
  source_system,
  event_type,
  event_value,
  CASE
    WHEN event_value IS NULL THEN 'UNKNOWN'
    WHEN event_value >= 200 THEN 'CRITICAL'
    WHEN event_value >= 25 THEN 'REVIEW'
    ELSE 'NORMAL'
  END AS priority
FROM raw.training_events
WHERE source_system IN ('microgrid_poc', 'sensor_gateway')
ORDER BY event_value DESC, event_id ASC
LIMIT 8;
```

Résultat :

Challenge autonome réussi.

Capture : `J12-09\_autonomous-case-business-challenge.png`

12. Script reproductible et validation psql -f

Script :

```
`documentation/semaine-02/jour-12/code/J12_distinct_expressions_case.sql`
```

Commande :

```
psql -U postgres -d sql_training -f ".\documentation\semaine-02\jour-12\code\J12_distinct_expressions_case.sql"
```

Validation observée :

- dataset initial : 15 lignes, 4 sources, 6 types, IDs 1 à 18;
- 4 sources distinctes;
- 6 types distincts;
- 6 couples source/type;
- expressions et arrondis conformes;
- classifications CASE conformes;
- KPI : LOW=6, HIGH=5, MEDIUM=4;
- challenge : 8 lignes;
- contrôle final : `15 | 1 | 18`;
- aucune erreur SQL bloquante.

Captures :

- `J12-10\_distinct-case-script-validation-1.png`
- `J12-10\_distinct-case-script-validation-2.png`
- `J12-10\_distinct-case-script-validation-3.png`

13. Erreurs, observations et résolutions

Aucune erreur SQL bloquante n'a été rencontrée.

Observations :

- `DISTINCT` agit sur le résultat, pas sur les données stockées;
- `DISTINCT` multi-colonnes déduplique des combinaisons;
- `ROUND` améliore la présentation des résultats numériques;
- l'ordre des branches `WHEN` peut modifier le résultat;
- une branche `IS NULL` explicite rend la logique métier plus robuste;
- `CASE` permet de construire une couche sémantique à partir de données brutes.

14. KPI de progression

15. Durée et pilotage

- Durée prévue : 90 minutes.
- Durée réelle : 62 minutes.
- Timer arrêté avec 28 minutes restantes.
- Écart prévu/réel : -28 minutes (62 min réalisées pour 90 min prévues).
- La durée de J12 est définitivement arrêtée ici ; les opérations Git ultérieures ne modifient pas cette mesure.

16. Potentiel de réutilisation

J12 peut devenir :

- tutoriel : « DISTINCT et CASE dans PostgreSQL »;
- vidéo : « Transformer des données brutes en catégories métier avec CASE »;
- exercice sur les frontières de classification;
- atelier `NULL` / `CASE`;

- démonstration `CASE + GROUP BY` pour produire des KPI;
- module Analytics Engineering sur la création d'une couche sémantique simple.

17. Conclusion

J12 marque le passage de la simple sélection de données à leur transformation en information métier. `DISTINCT` permet de maîtriser l'unicité du résultat, les expressions créent des mesures dérivées, et `CASE` transforme des valeurs brutes en catégories exploitables pour l'analyse.

Le script reproductible a été validé avec `psql -f` et le dataset est resté intact.

18. État final du rapport et opérations hors timer

1. placer les rapports MD/DOCX/PDF dans le dossier J12;
2. vérifier les captures et le script;
3. `git add` du dossier J12;
4. contrôler `git status`, `git diff --cached --stat` et `git diff --cached --name-status`;
5. commit prévu : `docs: complete J12 SQL DISTINCT expressions and CASE training`;
6. push vers `origin/main`;
7. confirmer `HEAD -> main, origin/main` et un working tree propre;
8. conserver la durée J12 définitivement arrêtée à 62 minutes ; ne pas ajouter le temps Git.

Le rapport pédagogique J12 et sa mesure de durée sont clôturés ici. Les opérations Git peuvent être réalisées ensuite comme traçabilité technique, sans modifier la durée réelle de 62 minutes.